```
do {
 statements
} while (condition);
```

Rethinking the last code sample, we might want to make sure that the LED is off if the switch is off before continuing on with the rest of our sketch. Using the `do…while` loop we could write the statement like the following:

```
do digitalWrite(13, LOW);
while (digitalRead(2) == LOW);
```

In this sample, no matter the condition, the `do` statement will turn off the LED on pin 13 at least once and then test the input pin 2. While the expression `digitalRead(2) == LOW` remains true, the `do` loop will repeat infinitely, keeping the LED off. When the condition is no longer true, the `do` loop will exit and return to normal program flow.

Switch

The `switch` statement is like a really nifty version of the `if` statement in that it can execute one or more blocks of code, depending on a range of conditions. Basically, the `switch` statement compares the value of an expression against a list of cases executing whatever code begins after that case when a match is found. `switch` is a fairly powerful and complex control structure so we will only scratch the surface here, revisiting it in later chapters when we can better apply it. The basic syntax of the `switch` statement follows:

```
switch (expression) {
  case constant:
    statements;
  case constant:
    statements;
  default:
    statements;
}
```

The number of cases can be as many as you need (or have the memory for), but each one should have a unique constant or single-byte character following it—variables like `myValue` are not allowed as a case. After the colon that defines the case, we can have a block of code that is executed when that case is true. The default case allows for code to be executed when a specific case has not been previously specified.

To get a better idea for how the `switch` statement works, let's say we want to modify the early LED blink sketch using the switch statement so that when the tilt switch is activated, the LED will stay on rather than blink. To do that we could use the following code sample:

```
switch (digitalRead(2)) {
  case 0:
    digitalWrite(13, LOW);
    delay(1000);
  case 1:
    digitalWrite(13, HIGH);
    delay(1000);
}
```